

1. System Overview

The platform operates as a testbed designed to evaluate C++ exchange matching engines. Participants are tasked with implementing the core matching logic. The platform wraps their implementation in a wrapper, compiles it within a secure sandbox, and evaluates it across two dimensions: correctness (Phase 1) and hardware performance (Phase 2).

Repository Structure

- `core/`: Compiled inside the Docker sandbox
- `worker/`: Node.js sandbox worker for job execution.
- `bot_node/`: Bot fleet manager for correctness load generation.
- `gateway/`: Express.js API backend and React (Vite) frontend dashboard.
- `shared/`: Shared utilities for Redis state management and telemetry parsing.
- `scripts/`: Build scripts and market orders generator.

2. The Contestant Interface

Participants interact with the platform entirely by implementing a function that returns a pointer to an `ExchangeEngine` class. The platform handles all socket I/O and telemetry.

The `ExchangeEngine` Class Contestants must implement the following methods:

- `Init()`: Called once at startup to pre-allocate memory pools.
- `Clear()`: Called between Phase 2 probes (explained in sec 5) to reset the engine state.
- `ProcessOrder(const Order &order)`: The critical hot path.
- `GetRestingOrders(uint16_t instrument_id)`: Returns the final book state for Phase 1 verification.

Required Logic

- **Matching**: Orders match strictly on Price-Time Priority.
- **Cancellations**: Existing resting orders must be removed via their `sequence_id`.
- **Self-Trade Prevention**: Orders that would execute against resting liquidity belonging to the exact same `client_id` must be rejected immediately.
- **Duplicate Detection**: Orders with duplicate `sequence_id` values must be dropped.
- **Trade Reporting**: Upon every match, the engine must synchronously call the `Telemetry::ReportTrade()` function.

3. The Wrapper

The platform wraps the participant's engine in a two-thread architecture designed to isolate the network stack from the matching logic.

The 32-Byte Wire Protocol To maximize L1 hardware cache prefetching, the `Order` struct is padded to get exactly 32 bytes, allowing 2 orders to fit into a 64-byte CPU cache line.

- `sequence_id` (8B)
- `price` (8B)
- `quantity` (4B)
- `client_id` (4B)
- `instrument_id` (2B)
- `side` (1B)
- `type` (1B)
- `_pad` (4B) - Explicit padding for strict alignment.

Thread Model & Lock-Free Data Structures

- **Network Thread:** Utilizes non-blocking OS multiplexing (epoll/kqueue) to accept TCP connections, parse 32-byte orders, and push them to an internal queue.
- **Dispatcher Thread:** Pulls from the queue, tracks execution latency, and invokes `engine->ProcessOrder()`.
- **MPSC Ring Buffer:** Communication between the network and dispatcher is handled by a Vyukov bounded Multiple-Producer-Single-Consumer lock-free queue with a statically allocated capacity of 8,388,608 slots (~256 MB).

4. Evaluation Phase 1: Correctness

Phase 1 guarantees that an engine is functionally correct before it is benchmarked.

Objective & Execution The engine is subjected to 10 seconds of non-deterministic traffic generated by a C++ Bot Fleet. The focus is entirely on uncovering race conditions, memory leaks, and logic violations.

Bot Fleet Composition It deploys 32 concurrent TCP connections executing 8 distinct trading archetypes:

- **Market Makers:** Post deep bid/ask ladders and drift mid-prices.
- **Aggressive Takers:** Cross the spread to sweep liquidity.
- **Cancel Spammers:** Rapid-fire invalid IDs and double-cancels.
- **Wash Traders:** Attempt to force illegal self-matches.
- **Flood Bots:** Inject edge-case prices.
- **Layering Bots:** Stack deep liquidity and bulk-cancel to force tree rebalancing.

- **Hot Quote Bots:** Rapid cancel-and-replace loops.
- **Burst Injectors:** Simulate extreme microsecond market shocks.

Verification We use a single-threaded Golden Model.

- Every order received by the container is logged to `input_ledger.bin`.
- Hashed logs of every trade execution (`HashTrade`) and the final order book state (`HashSortedBook`) is also logged.
- The Golden Model replays `input_ledger.bin`. If the output hashes differ from the Golden Model by a single byte, the submission fails and Phase 2 is skipped.

5. Evaluation Phase 2: Microsecond Benchmarking

Submissions that achieve correctness advance to Phase 2.

The Pre-Computed Workload Contestants are fed a sequence of pre-compiled `nasdaq_{N}x.bin` files. These files contain millions of typed 32-byte structs that have exact nanosecond dispatch delays, ensuring every participant faces the exact same market (barring few network jitters).

The Capacity Search Algorithm To locate the precise breaking point of the engine, we utilise a 2-pass algorithm:

- **Coarse Pass:** Scales throughput (4x, 8x, 12x, 16x, 20x, 24x, 28x, 32x) of a 20,000 Ops baseline. Holds each level for a 10 sec window until **failure** (defined below).
- **Fine Pass:** Executes a linear search within the established bracket (eg: if 20x failed, then 17x, 18x and 19x would be tried sequentially).
- We allow up to 3 retries for every probe to account for VM noise.

Health Checking Wrapper tracks health internally. Every 1,000 orders, it checks the current queue depth and the processing time. If `queue_depth` becomes greater than a pre-defined threshold or time taken to process an order takes time greater than a pre-defined threshold for 3 consecutive checks, a health breach is declared.

6. Scoring Engine & Leaderboard Mechanics

The final score is a composite metric weighted toward raw throughput (orders processed per second), but penalized by p99 latency.

The Scoring Formula:

`Score = floor ( max_ops / ( 1 + ln ( 1 + p99 / 1000 ) ) )`

- **Throughput (max_ops):** The peak 1-second order processing rate achieved at the highest passing multiplier.

- **Latency (p99):** The 99th percentile order processing time in nanoseconds.
- **Logarithmic Penalty:** The logarithmic function reduces the volatility of extreme latency outliers while maintaining meaningful differentiation between sub-microsecond engines (e.g., 600 ns vs. 900 ns).

7. Distributed Architecture

Physical Separation

- **The Control Plane (General Compute):** The Express.js API Gateway, React frontend serving, and the Redis state store live on standard cloud instances (e.g., AWS t3/m5 instances). These handle human-facing UI, job enqueueing, and leaderboard state.
- **The Worker Plane (Compute-Optimized Sandbox):** The Node.js worker scripts run on dedicated compute-optimized nodes (e.g., AWS c6g/c7i). This node is responsible for compiling the C++ code and managing the Docker container.

Native Binaries: The **bot_replay** (Phase 2 paced load generator) and **verifier** (Phase 1 Golden Model) are native C++ binaries that execute directly on the worker's host OS, *outside* the Docker container. This allows them to interact with the containerized engine over the local TCP loopback without much overhead.

- **The Storage Layer (AWS S3):** Acts as a decoupled storage bridge. The Control Plane uploads raw **.cpp** files here, and the Worker Plane downloads them.

Container Orchestration

- **Bridge Networking:** The IaC puts the core microservices onto a dedicated Docker bridge network (**hft-network**). This ensures internal C++ TCP traffic during phase 2 bypasses the host OS's hypervisor translation layer, eliminating network jitter.
- **Docker-out-of-Docker (DooD) Topology:** The Sandbox Worker is provisioned with a **/var/run/docker.sock** mount. Instead of running Docker *inside* the worker, it acts as a manager that signals the host OS to spawn ephemeral contestant sandboxes as sibling containers.

Horizontal Scaling Strategy

- **Worker Auto-Scaling:** The Node.js workers use Redis **BRPOP** to listen to the **hackathon:queue**. Because workers operate statelessly pulling from S3, the worker pool can be horizontally scaled using orchestrators like Kubernetes
- **Process Isolation:** Only one Docker container is executed per physical Worker Node at any given time to ensure the engine has absolute control over the hardware caches and memory bandwidth.

The Bot Plane: The `bot_node` operates as a separate orchestrator listening to the `hackathon:bot_queue`. It spawns the massive, 32-thread `bot_fleet` C++ binary used during Phase 1.

Resource Limits and Security: The Docker runtime enforces strict boundaries:

- **Hardware Quotas:** `--cpus=4` and `--memory=4096m..`
- **Timeout & Read-Only Core:** The container is hard-killed after 8 minutes (`DOCKER_HARD_LIMIT_MS`). The trusted C++ infrastructure (`/core:ro`) is mounted as read-only to prevent runtime injection.

8. Improvements

If given more time and bare-metal resources, we would implement the following architectural upgrades:

1. **Hardware Isolation (`isolcpus`)** Currently, in Docker's `--cpus=4`, the dispatcher thread can be held off to handle kernel level jitters while the clock on the `ProcessOrder()` is still running.

Improvement: Deploy on bare-metal Linux instances and use the `isolcpus` flag to completely remove specific physical CPU cores from the OS scheduler. Contestant engines would be strictly pinned via `cpuset` to these isolated cores, achieving zero context-switching jitters.

2. **TLB Miss Mitigation:** The C++ core currently pre-allocates an 8,388,608-slot `MpscRingBuffer`, which consumes roughly 256MB of static RAM. Under standard Linux 4KB memory pages, this requires over 65,000 page table entries. As the engine cycles through the ring buffer, the CPU's TLB will constantly miss, forcing expensive page walks.

Improvement: Configure the bare-metal Linux host to allocate **1GB HugePages**. By mapping the container's memory space to a single 1GB HugePage, the entire ring buffer fits within a single TLB entry. This eliminates TLB misses on the hot path and significantly tightens the p99 latency spread